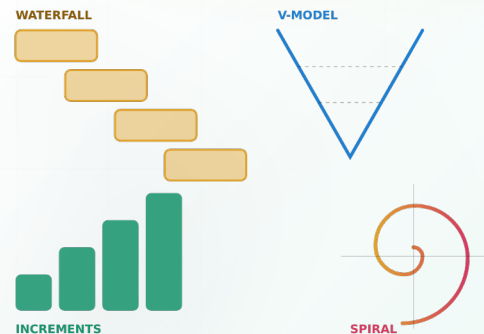


Plan-Driven Process Models II: Increments, the Spiral and Choosing

Three ways of moving the bad news earlier — building in slices, delivering in slices, and letting risk decide the order — then a heuristic for choosing a process, applied part by part to SANTE-BASSAM.

DATE**Friday, 18 September 2026****MEETING TIME****10:00 - 11:20 AM****ROOM FORMAT****Lecture + process-selection exercise**

04

2 models worked on SANTE-BASSAM**1** model assigned to you**6** risks sized**4** spiral loops**7** figures

0

Where Session 3 left us

Waterfall found the wrong search requirement in week 12 at a cost of 9 team-days; the V-model found a hole in the test plan in week 2 but still met the

clerk in week 12. Today's three models are three different ways of **meeting the clerk earlier** — and the session ends with how to choose.

SESSION 3

Waterfall · V-model

Risk lands at the end.
Evidence is signed. 14 tests written across four levels.

TODAY

Incremental development

Integrate slice by slice — the team learns from itself, not from the customer.

TODAY · ASSIGNED

Incremental delivery

Deploy each slice for real use. Defined here; worked by you as homework.

TODAY

Spiral · choosing

The largest unretired risk decides what you build next; a decision heuristic for real projects.

1 Learning outcomes

L0 4.1 Distinguish incremental *development* from incremental *delivery*, and say what each lets the team learn before the end.

L0 4.2 Order increments by risk, not by ease of building, and justify the order for SANTE-BASSAM.

L0 4.3 Describe the four spiral quadrants, compute risk exposure, and use the largest unretired risk to decide what to build next.

L0 4.4 Given a project description, select a process model and defend it against a named alternative using three project characteristics.

L0 4.5 Draw the line in SANTE-BASSAM between what is specified once and what is delivered iteratively.



How this session is built. Incremental development and the spiral are worked in full on SANTE-BASSAM. **Incremental delivery is defined but left for you** — it is this week's homework, and you have four worked examples (two from Session 3, two from today) to model your answer on.

3.1 Incremental development

One rule: build the system as a sequence of working, integrated slices — each with its own small specify → design → build → test cycle — but *release nothing until the whole is complete*. The team learns from integration; the customer sees nothing until the end.

FIXES IN WATERFALL

Integration is no longer one event in week 11. Each slice integrates with the last, so interface mismatches surface early and small.

DOES NOT FIX

Requirements risk. If the clerk needed search by mother's name, the team still learns it at the end — she hasn't touched the system.

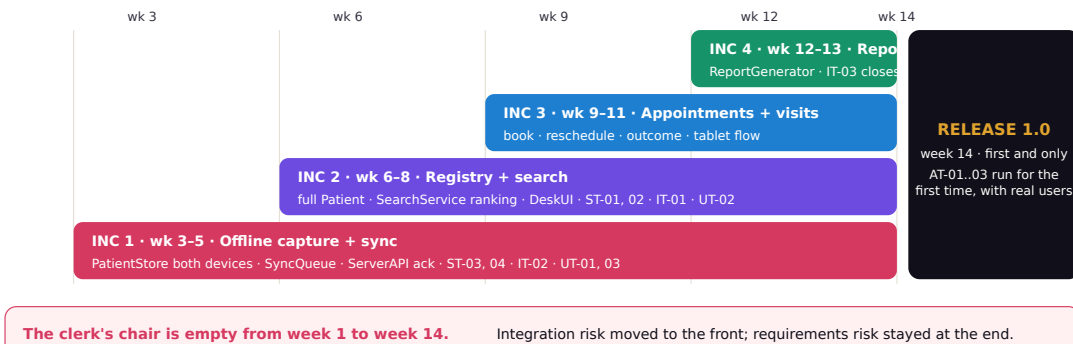
ORDERING RULE

Risk first, then value. The slice that answers your scariest technical question comes first — not the easiest or prettiest one.

APPLIED TO SANTE-BASSAM

Four increments, risk-first — nothing shipped until week 14

FIG. 1 · THE INCREMENTS STACK; EACH INTEGRATES WITH EVERYTHING BEFORE IT; ONE RELEASE AT THE END



LEARNED BY WEEK 8 (WATERFALL: WEEK 11)

- **Sync contract was wrong** — the desk needs the full record back on ack, not just the id. Two hours, not two days.
- **Card numbers aren't unique** — the post restarted numbering in 2023. Card number becomes an attribute, never the key.
- **Tablet ranks 10k records in 4 s** (laptop: 0.6 s). Fixed with a prefix index inside the increment.

STILL UNKNOWN IN WEEK 13

- Whether the clerk will use it.
 - Whether the district officer trusts the report.
 - Whether "mother's name + village" is even the right search.
- ...because no clerk has touched the system.

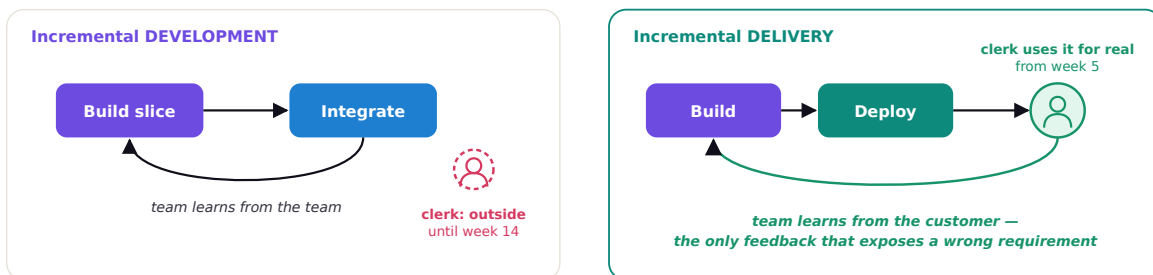


Carry forward. Incremental development is a way of *building*. It changes what the team learns from itself. It does not change what the team learns from the customer, because the customer is not in the loop.

3.2 Incremental delivery

One rule: each increment is not just integrated but *deployed to real users and put into service*. First delivery = the highest-priority core; every later one adds to a system already in daily use.

FIG. 2 · THE ONLY DIFFERENCE IS WHO IS INSIDE THE FEEDBACK LOOP



THE COST

Every delivery is a release: install, migrate data, training note, support, and a way back if it fails. Four deliveries cost more than one.

WHAT IT BUYS

The ability to be wrong cheaply. A wrong requirement in delivery 1 is corrected in delivery 2, with ten weeks left and the code still small.

THE DISCIPLINE

Each delivery must be a *usable whole*. "Registration without search" is not deliverable — the clerk would register patients she can never find again.

ASSIGNED — NOT WORKED

Incremental delivery for SANTE-BASSAM is this week's homework

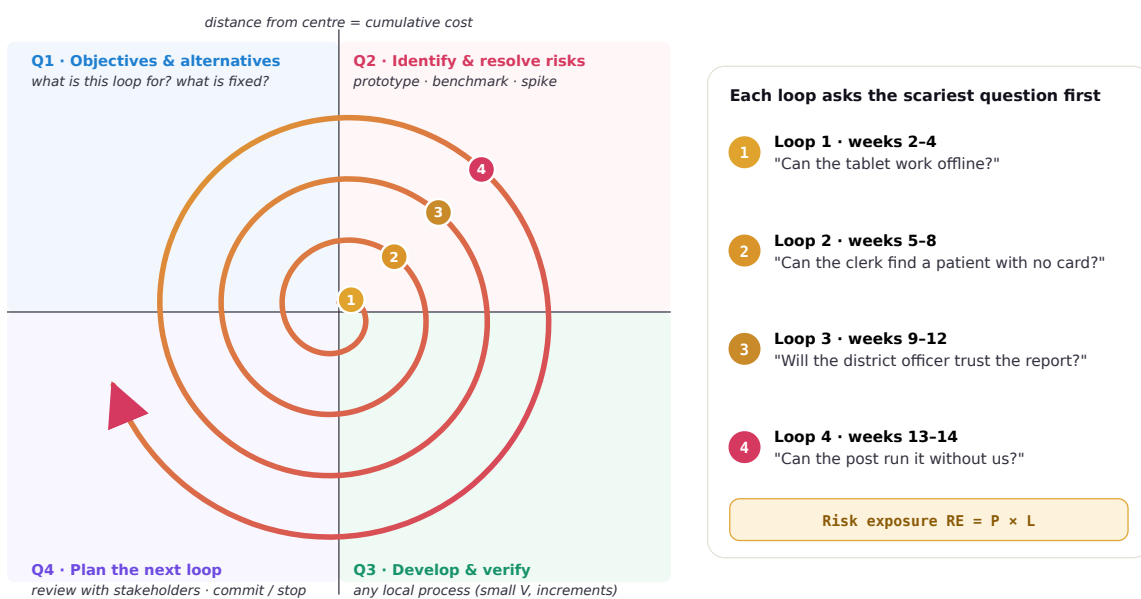
Use the four increments in Fig. 1 as raw material, but do not copy them. Ask of each: **could the health post actually use this on its own for three weeks?**

If not, it is an increment, not a delivery. Full brief in section 5.

3.3 Boehm's spiral: process driven by risk

One rule: the largest unretired risk decides what you build next. The spiral (Boehm, 1988) is a *meta-process*: each loop passes through four quadrants, and distance from the centre is cumulative cost.

FIG. 3 · FOUR QUADRANTS, AND THE FOUR SANTE-BASSAM LOOPS WE WORK THROUGH



Quadrant 2 is quantitative: for each risk, estimate probability **P** (0-1) and loss **L** (team-days). The risk with the largest **RE = P × L** is what the loop works on — even when it is the least visible thing the team could build.

APPLIED TO SANTE-BASSAM

The risk register that drives the loops — then the loops themselves

FIG. 4 · RISK REGISTER BEFORE LOOP 1, SORTED BY EXPOSURE (BUDGET ≈ 280 PERSON-DAYS)

R1	Offline sync can't be made reliable on the tablet	.4 × 60	24
R2	Clerk can't find returning patients (no card, duplicates)	.5 × 30	15
R3	District officer distrusts the report, keeps hand-counting	.5 × 20	10
R4	Power failures corrupt the local database	.3 × 25	7.5
R5	Team can't reach the post often enough for feedback	.4 × 15	6
R6	Legal constraints discovered late	.2 × 20	4

R1 wins loop 1 — the least visible thing the team could build. R4 rides along; the same spike retires both.

LOOP 1 · WK 2–4

Can the tablet work offline?

- Q1** (a) SQLite + own queue · (b) CouchDB/PouchDB · (c) no local store
- Q2** Thinnest register-offline-then-sync path for (a),(b); pull the plug 20× each. (a) 0 lost once journaling on; (b) 1.8 s saves; (c) rejected
- Q3** SyncQueue, PatientStore, ServerAPI ack — UT-01, 03, IT-02, ST-03 pass (a small V inside the loop)
- Q4** Demo: register, pull plug, restart, sync. **R1 → P 0.05. R4 retired. Next: R2.**

LOOP 2 · WK 5–8

Can the clerk find a patient with no card?

- Q1** (a) exact name · (b) fuzzy name + village + birth year · (c) photo on record
- Q2** Paper prototype, 50 real cards, 10 lookups each. (a) fails 4/10; (b) 10/10 — **and the clerk volunteers "I always ask the mother's name"** → SR-1.1 in week 5, not 12
- Q3** SearchService, DeskUI, full Patient, register migration — two short increments
- Q4** Clerk uses it one morning. R2 retired. New risk R7: duplicate entries (RE 5). Next: R3.

LOOP 3 · WK 9–12

Will the district officer trust the report?

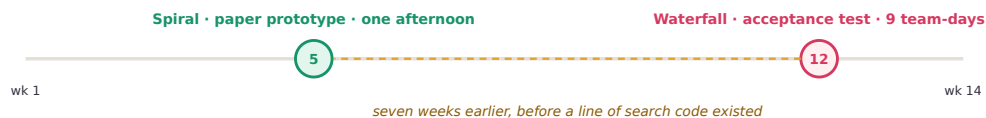
- Q1** (a) fixed monthly table · (b) exportable spreadsheet · (c) both
- Q2** Mock report from one real month; "what would you check before signing?" She hand-checks two villages → (c): the export is her audit trail
- Q3** Appointments, visits, ReportGenerator + export; ST-05, UT-04 and the missing IT-03. AT-03 drafted with her in week 9
- Q4** AT-03 on October data. R3 retired. Next: R5 (handover).

LOOP 4 · WK 13–14

Can the post run it without us?

- Q1** Clerk registers 10 real patients, nurse books 5 appointments, team absent
- Q2** Risk: nobody at the post can restart sync after a failure
- Q3** One-page operations sheet + a "sync now" button that reports in plain language
- Q4** Release 1.0. Training done. AT-01..03 signed.

FIG. 5 · WHAT THE SPIRAL CHANGED: THE WEEK "SEARCH BY MOTHER'S NAME" WAS LEARNED



The spiral did not make the team smarter. It made them ask the scariest question first, every four weeks, and spend money on the answer.



Carry forward. Every modern process, agile included, inherits the spiral's rule: the largest unretired risk decides what you build next. Teams that iterate without a risk register are not doing the spiral — they are doing waterfall in small pieces.

3.4

Choosing: a decision heuristic

Read the project's dominant characteristic; it points at a process. Real projects mix — a hospital may certify its clinical-rules module under a V while running its appointment front end in two-week sprints. **Choosing one process for the whole project because it is the one you know is a failure of judgement, not a simplification.**

Requirements stable and externally fixed (regulation, contract, standard)

Waterfall / V

Safety-critical; certification evidence required

V-model

Requirements genuinely unknown until users touch the system

Incr. delivery / agile

Novel technology; major technical uncertainty

Spiral – spike first

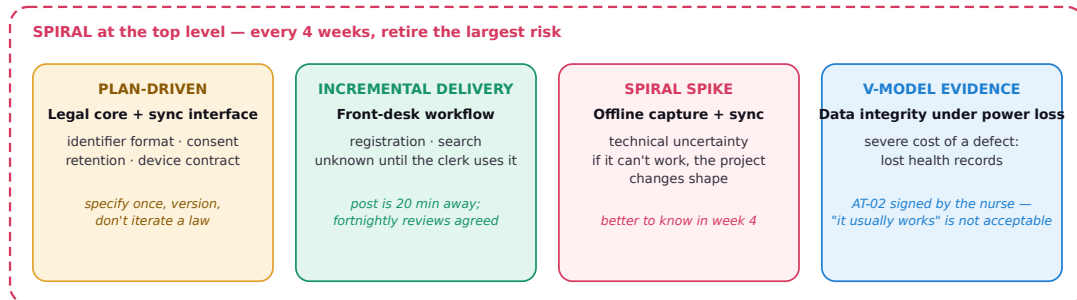
Large distributed team, multiple organisations, fixed interfaces

Plan-driven between, agile within

Small co-located team, accessible customer

Agile (Session 5)

FIG. 6 · ONE PROJECT, FOUR PROCESSES — THE POSITION YOUR PROJECT PLAN WILL DEFEND



Session 5 supplies the mechanics for the incremental-delivery part (it will call it agile).

4

In-class exercise · process selection

Groups of four, 15 minutes. Assign a process to each project; defend one to the class. Ask the four questions from today's translations: **How volatile are the requirements? How expensive is a late defect? What evidence must the project produce? What feedback exists before release?**

A

Crop-spraying drone

Flight-control software for an agricultural drone over cocoa plantations. A fault could injure workers. Civil-aviation certification required before commercial use.

B

Mobile-money savings app

Fintech start-up in Abidjan. Three competing hypotheses about what customers want; eight months of funding.

C**COBOL payroll replacement**

20-year-old system, 4,000 employees. New system must match the old exactly, quirks included. Parallel run for one full payroll cycle.

Trap: requirements "exist" — only as undocumented behaviour in old code.

D**Handwriting reader for field notes**

ML system reading agricultural extension officers' notes. Nobody knows whether the needed accuracy is achievable.

Trap: feasibility risk, not requirements risk — sprints without a kill criterion are not risk management.

**Chosen model**

+ three characteristics that drove it

**One named alternative**

and the specific reason you rejected it

**The largest risk**

your chosen process leaves unaddressed

**Which project is most like SANTE-BASSAM**

— and where the analogy breaks

5

Homework · incremental delivery for SANTE-BASSAM · due before Session 5 (Tuesday 22 September)

Pushed to your team repository as `s03-delivery-plan.md` plus the three D1 files. Produce the worked example yourself, to the standard of the four in this document.

**1 · The plan**

— 3 to 5 deliveries D1...Dn with a target week. For each: what is deployed; one sentence "After this delivery the clerk / nurse / CHW / officer can ..."; the acceptance test that must pass; the one question real use will answer. *Each delivery must be usable alone for three weeks.*

**2 · D1 artifacts**

— `D1-release-note.md` (what's in, what's known broken, who to call) · `D1-training-sheet.md` (one page the clerk can follow alone; hand-drawn screenshots fine) · `D1-feedback-and-rollback.md` (five questions after two weeks; exact steps back to paper)



3 • Compare

(250–400 words) — against the Fig. 1 plan: what does your team learn, and when, that the increment team does not? What does each delivery cost that an increment does not? Be specific to SANTE-BASSAM.



Reading

— Sommerville 10th ed., §2.1–2.3.
Optional: Boehm (1988), §1–3.



Assessment link. This plan becomes Section 2 of your Project Plan (due Session 10), alongside the process choice from 3.4 and its justification. Write it as if the nurse-in-charge will read it — because she will.

6

One-page recap

FIG. 7 • FIVE MODELS • ONE SHAPE • ONE RULE • ONE THING IT MOVES EARLIER



If you remember one sentence: **a process model is a decision about when the team is allowed to find out it was wrong.**

Key terms

waterfall model

phase sign-off

V-model

acceptance / system / integration / unit test

bidirectional traceability

traceability matrix

incremental development

incremental delivery

spiral model

risk exposure ($P \times L$)

spike / prototype

requirements volatility

process tailoring

Readings and references

1. Sommerville, I. *Software Engineering*, 10th ed. — Chapter 2, §2.1-2.3.
2. Royce, W. *Managing the Development of Large Software Systems*. Proc. IEEE WESCON, 1970.
3. Boehm, B. *A Spiral Model of Software Development and Enhancement*. IEEE Computer 21(5), 1988.
4. Pressman & Maxim, *Software Engineering: A Practitioner's Approach* — Chapter 2.
5. Forsberg, K. & Mooz, H. *The Relationship of System Engineering to the Project Cycle* (the V-model). NCOSE, 1991.

International University of Grand-Bassam · School of Science and Engineering · **CSC 4305**, Fall 2026

Next · Session 05 — Agile Development I: The Manifesto and Scrum